Compiler Architecture

Guillermo Polito











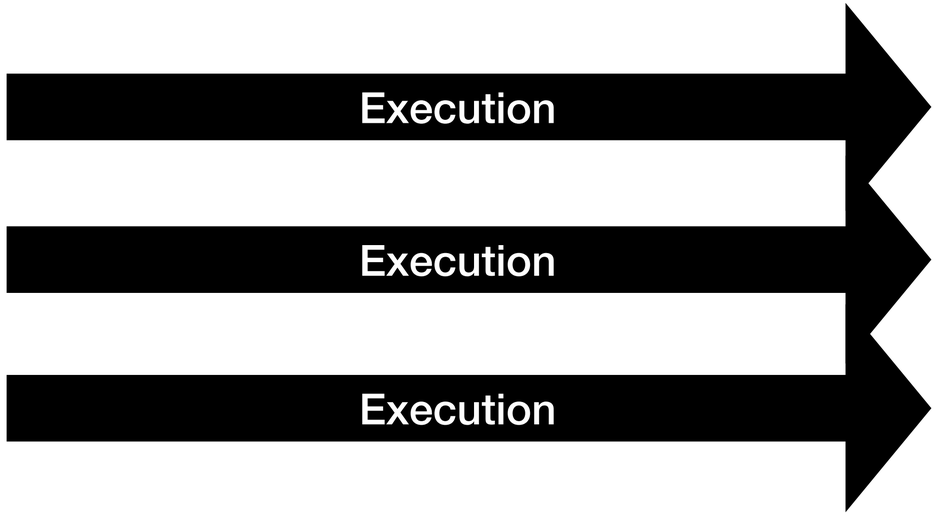


Goals

- Course introduction, initial concepts
- Understand compiler architectures
  - Phases
  - Intermediate representations
  - Common terminology

Interpreters

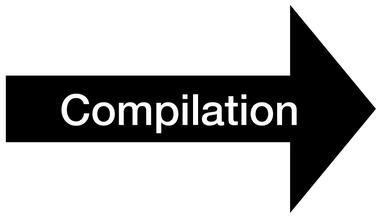
```
MyClass >> foo
    ^ 1 + 17
```



A program that executes a program in a *source* language step by step

Compilers

```
MyClass >> foo
    ^ 1 + 17
```




```
push 1
push 17
send +
returnTop
```




```
18
```

A program that translates a program in a *source* language to a *target* language

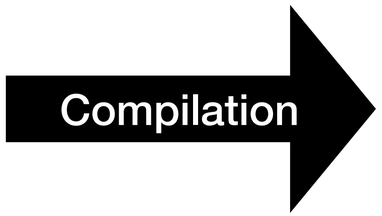
Interpreter vs Compilers - light overview

- Interpreters
  - High instruction dispatch overhead: jumping between instructions is done in *software*
  - Small optimization granularity: one small code chunk at a time
  - Fast startup
- Compilers
  - Large optimization granularity: functions, entire programs…
  - Compilation time can be expensive (see JIT compilers later)

Back to Compilers

Source language

```
MyClass >> foo
    ^ 1 + 17
```



Target language

```
push 1
push 17
send +
returnTop
```




```
18
```

A program that translates a program in a *source* language to a *target* language

Target Languages

- Another programming language => we talk about transpilation
  e.g., Pharo to C translation

- Some binary code for a virtual machine
  e.g., the Pharo or JVM bytecode

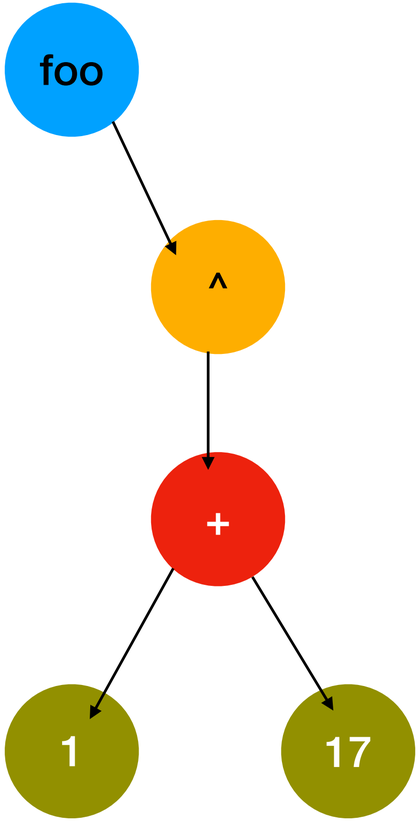
- Some binary code for a real machine
  e.g., machine code for x86, or ARMv8

Compiler Internals Example: Simple Compiler

Source code

```
MyClass >> foo
    ^ 1 + 17
```

Parsing



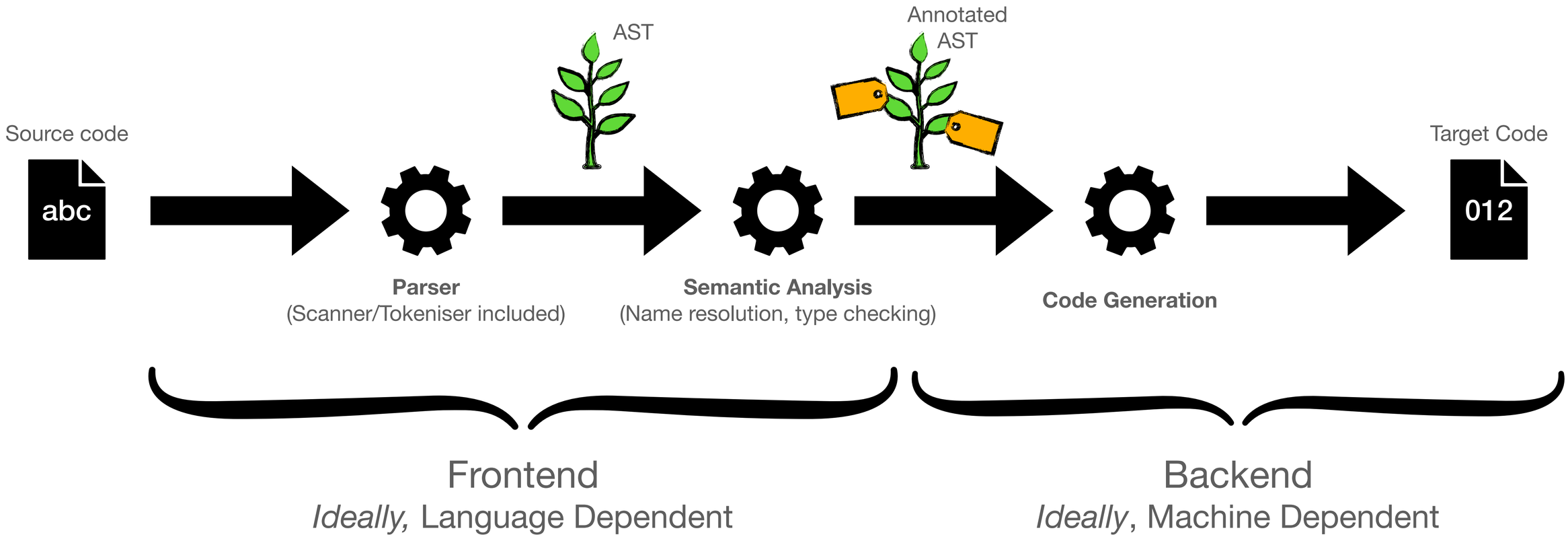
Intermediate Representation

Code Generation

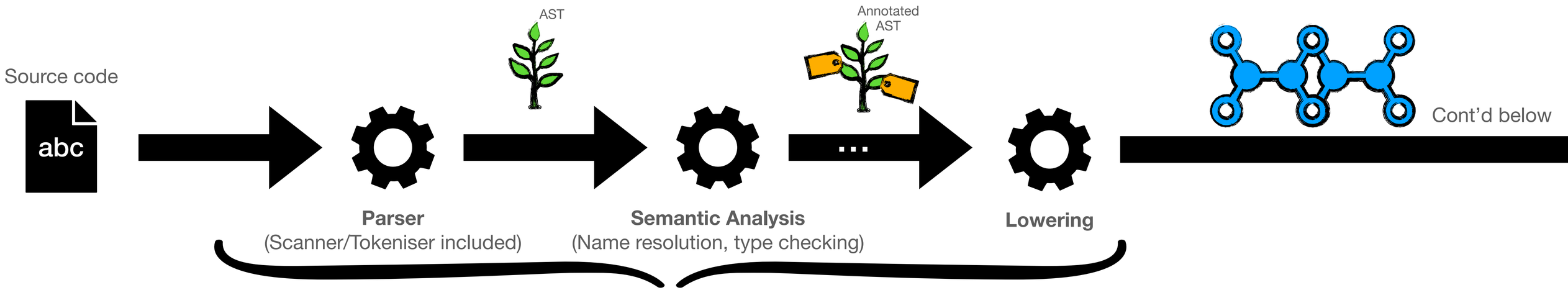
Target Code

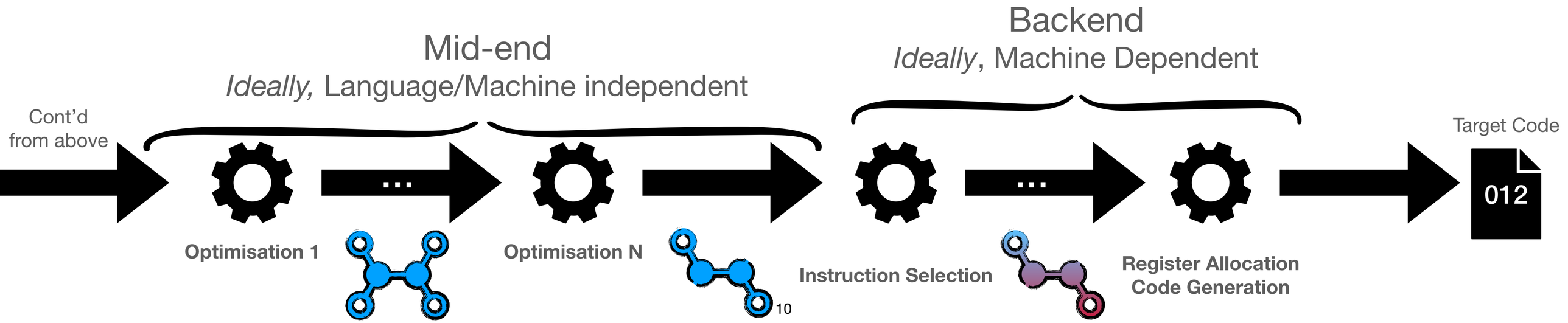
```
16r76
16r20
16rB0
16r7C
```

Compiler Phases by Example



Modern Optimizing Compiler Phases

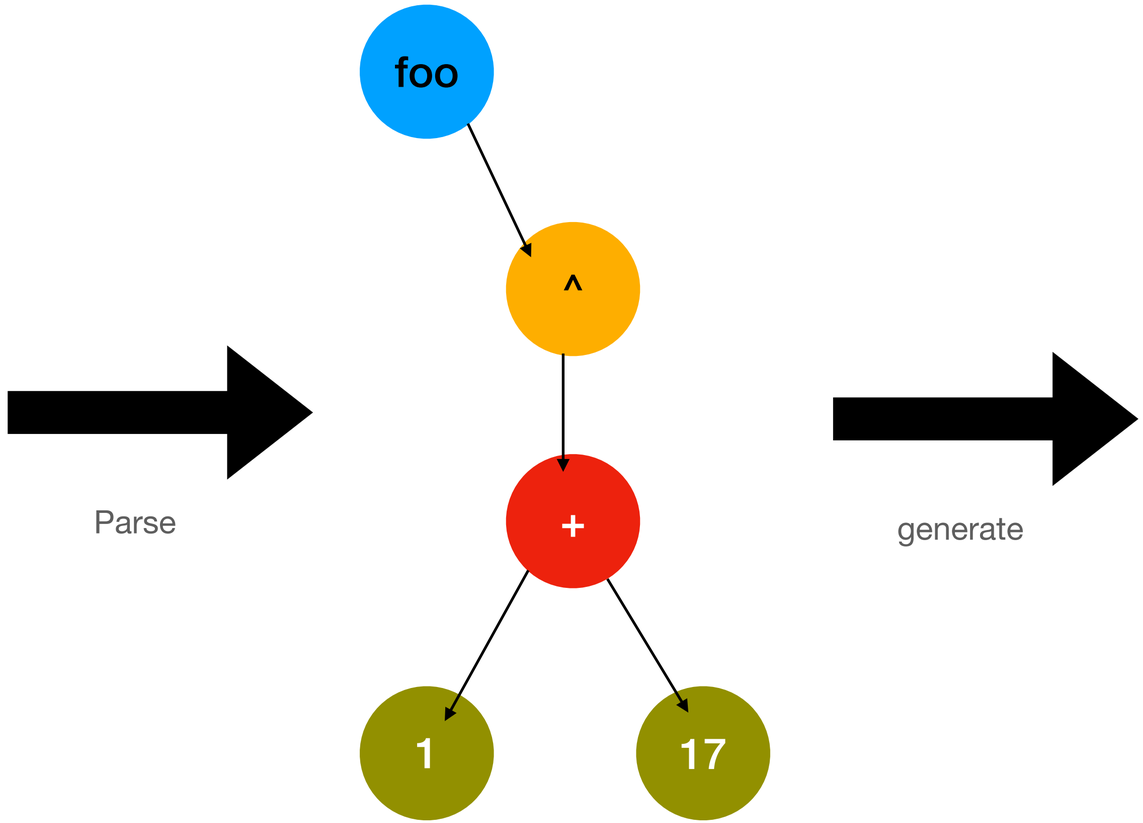




Real Example: The old Pharo Compiler

Source code

```
MyClass >> foo
    ^ 1 + 17
```



Parse

Intermediate Representation

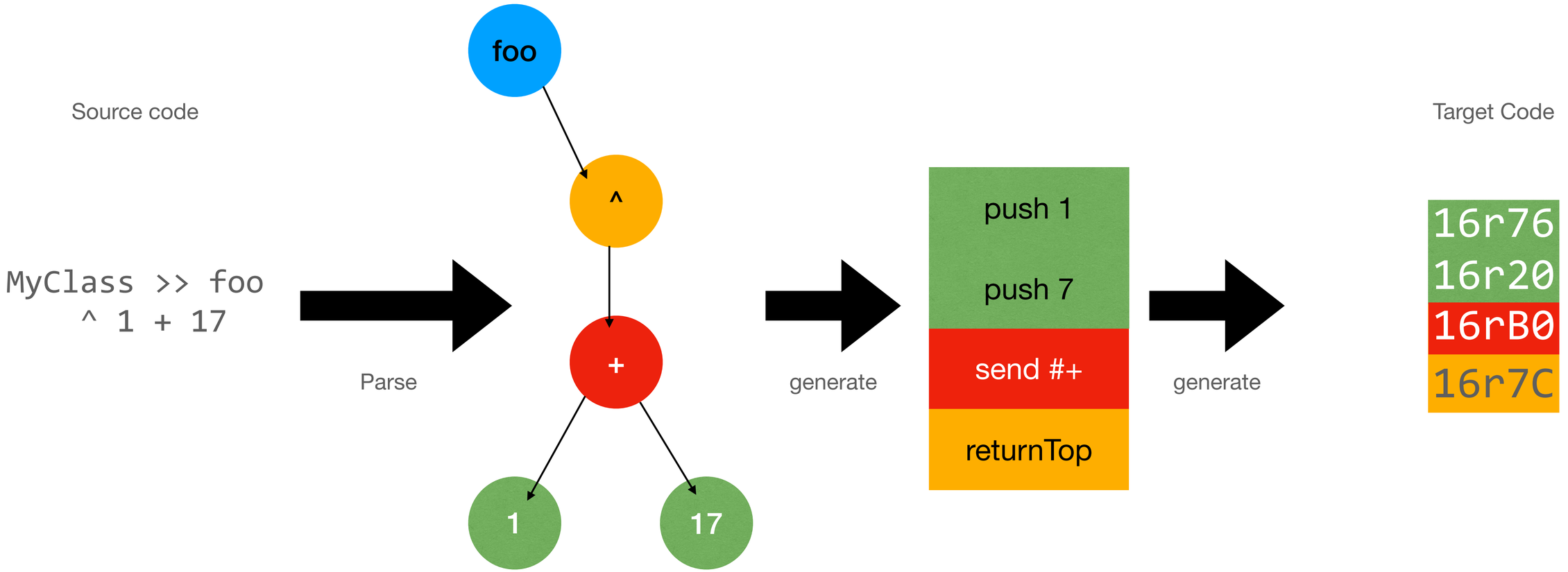
generate

Target Code

```
16r76
16r20
16rB0
16r7C
```

Real Example: The Pharo Opal Compiler

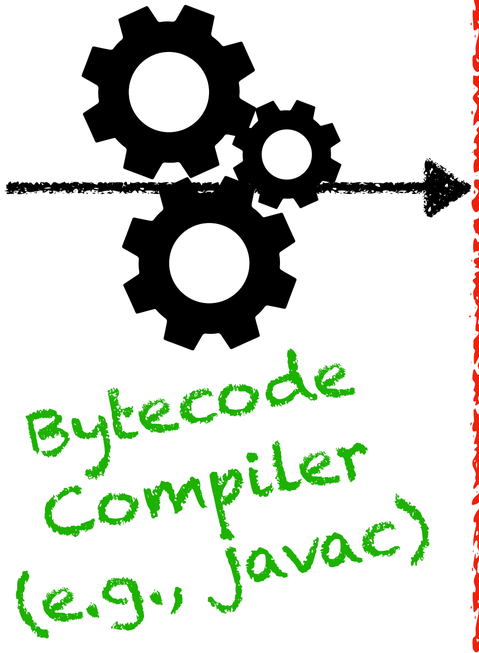
Introducing linear intermediate representations

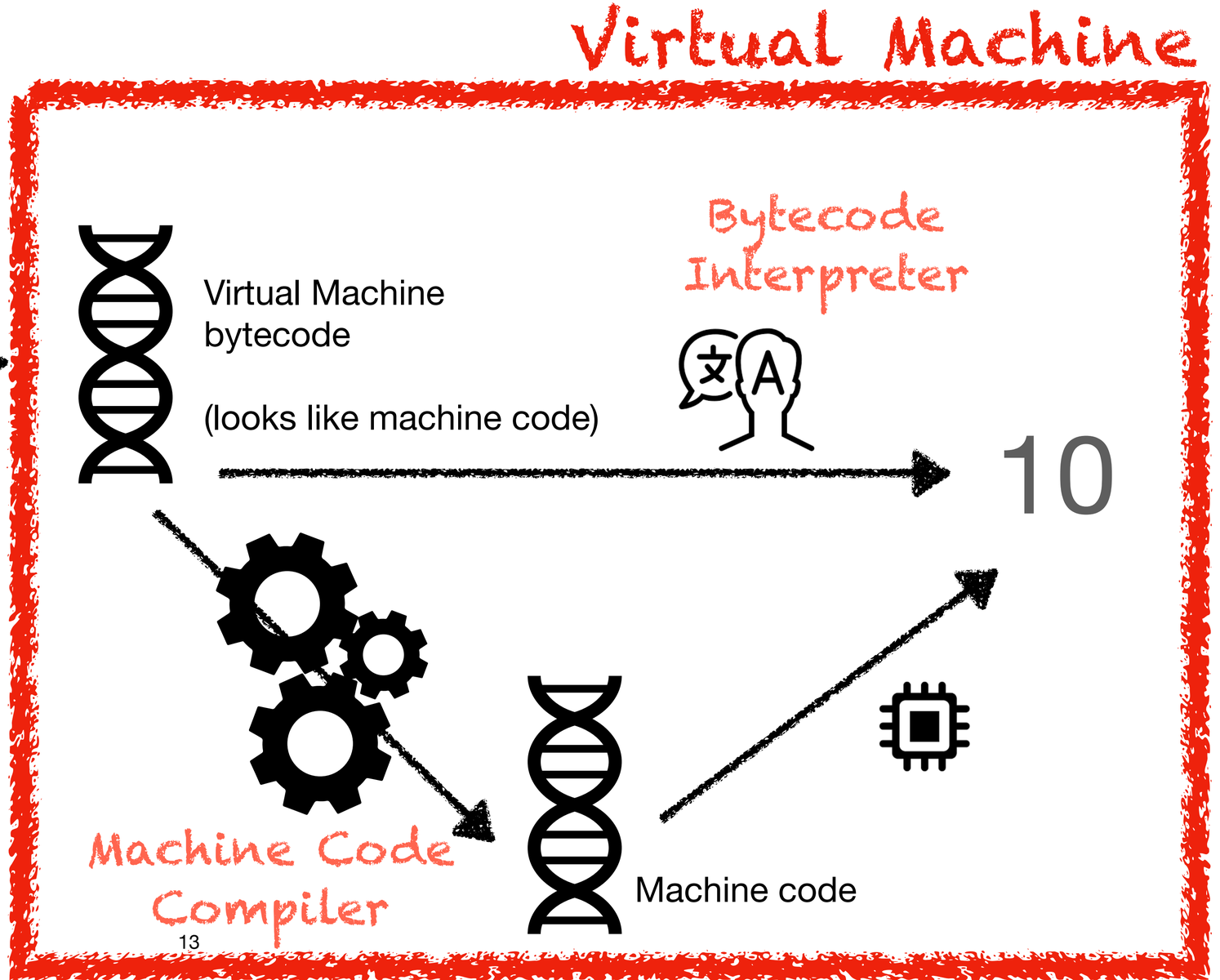


Modern Languages: Compiler*s* + Interpreters

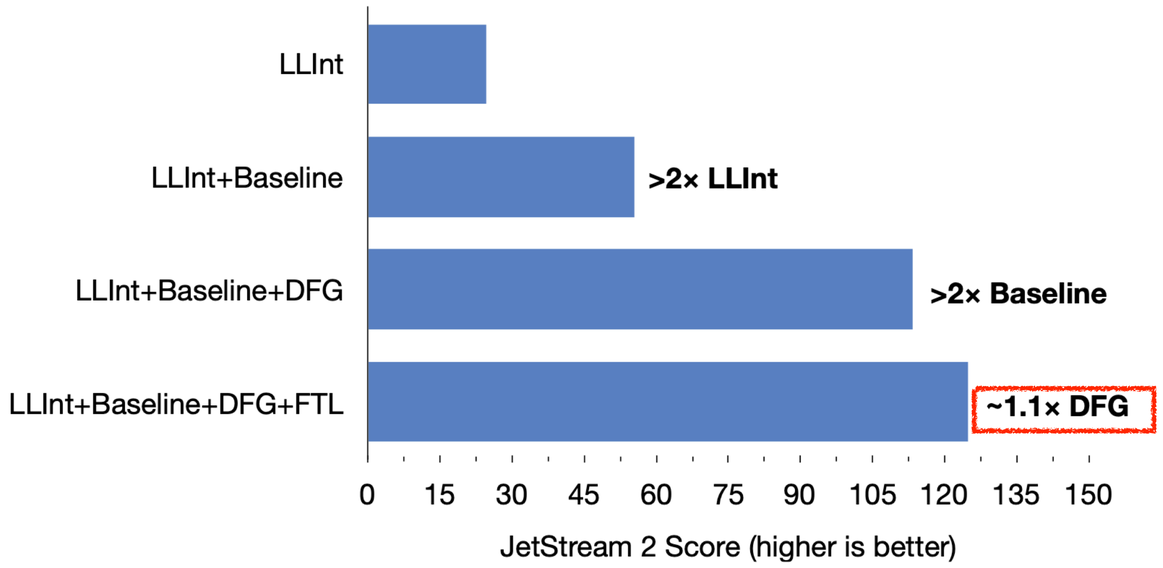
```
a := 1;
if (condition){
  a := a + 7;
}
return a + 2;
```







Javascript Core JIT Compiler



Parser

Bytecompiler

Generatorification

Bytecode Linker

LLInt

Baseline JIT

DFG Compiler

DFG Bytecode Parser

DFG Optimizer

DFG Backend

FTL Compiler

DFG Bytecode Parser

Extended DFG Optimizer

Lower DFG to B3

B3 Optimizer

Instruction Selector

Air Optimizer

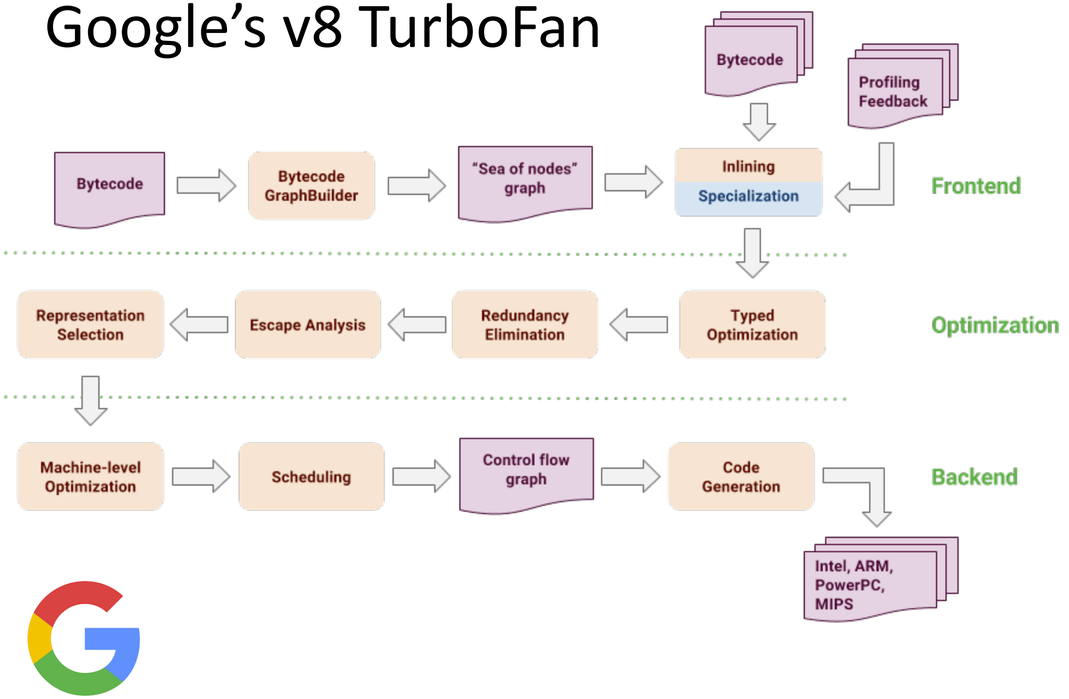
Air Backend

Multiple levels
Different representations
Engineering cost!

https://webkit.org/blog/10308/speculation-in-javascriptcore/

V8 and JSCore in depth

Google's v8 TurboFan



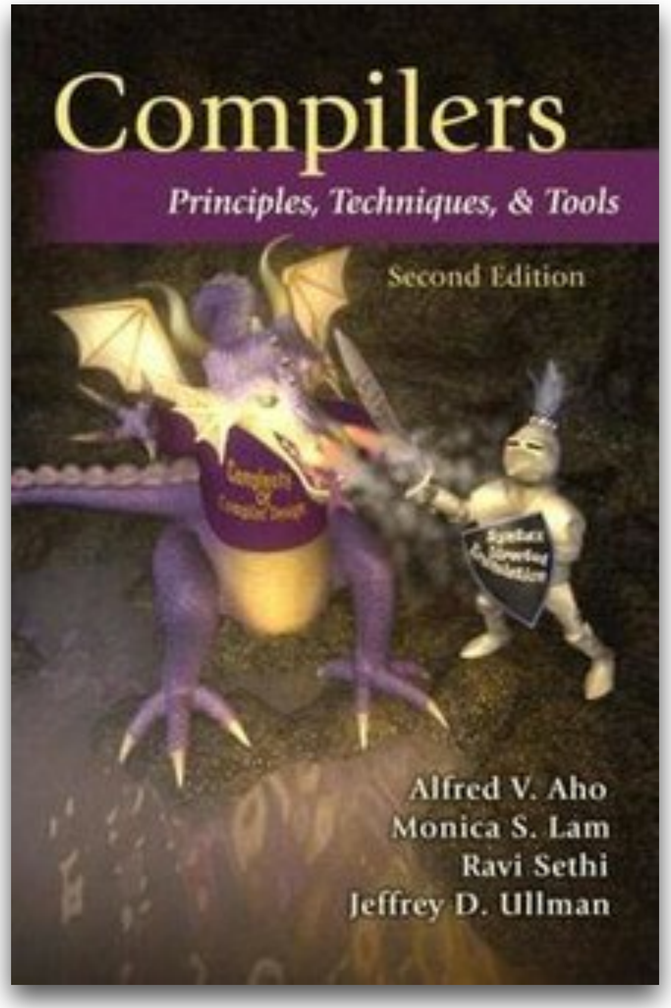
Apple's JavascriptCore[2021]



https://webkit.org/blog/10308/speculation-in-javascriptcore/
https://ponyfoo.com/articles/an-introduction-to-speculative-optimization-in-v8

Compiler Frontends

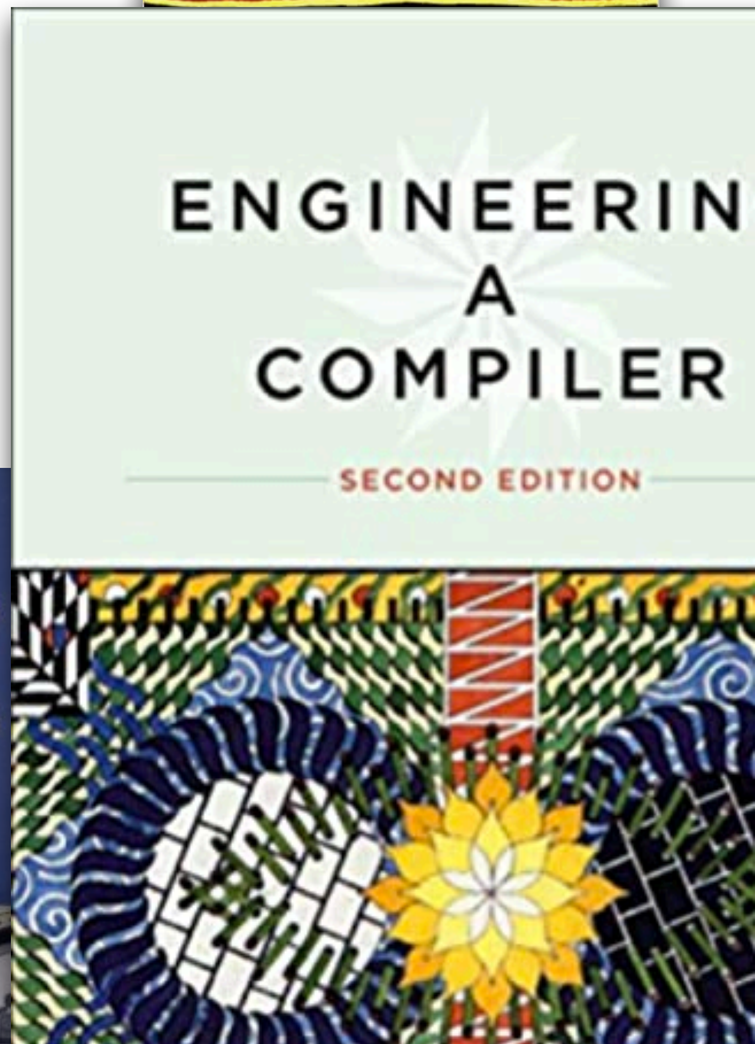
- ***Extensively*** studied in books
  - E.g., Parsers/Lexers/Scanners are >80% of the dragon book
  - Check it :)
  - The dragon book is not the only one

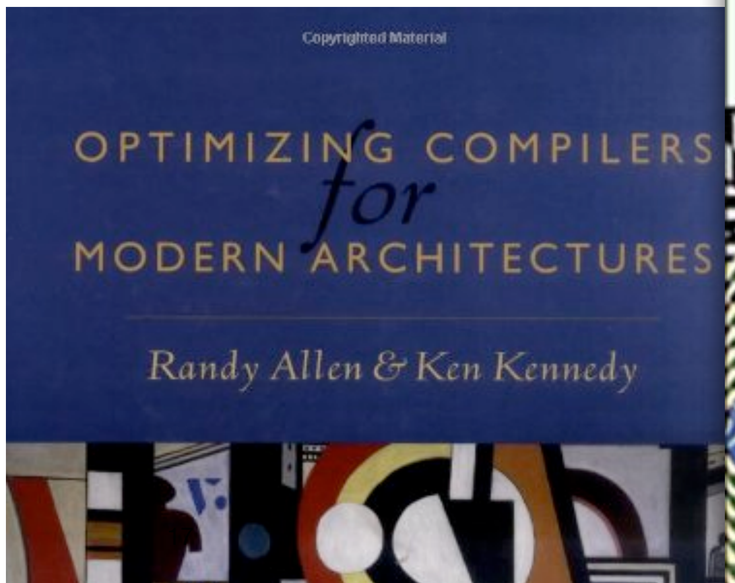


Compiler Mid- and Back- ends

- ***Fewer books*** covering these topics
  - Info is in research papers
  - Still an active area of research







A *dense and practical* domain

- Parsing Overview
- Stack vs register machines
- Assembly and Bytecode
- Code representations
- Code translation and transformation
- Code analysis
- Interpreters and compilers

A *dense and practical* domain

- Parsing Overview
- Stack vs register machines
- Assembly and Bytecode
- Code representations
- Code translation and transformation
- Code analysis
- Interpreters and compilers
- Call stack representation
- Optimizations
- Name analysis
- Register allocation
- Control flow representation
- Fuzzing and testing
- And languages, lots of languages

And it can get dens-*er*

But exists in reality

- Complex optimizations
  - e.g., loop parallelization
  - would require lots of classes each
- Different "programming" models
  - e.g., SIMD, tensors, GPU…
- Static type checking and type systems
  - They require a course on their own

- Complex implementations
  - LLVM, GCC, JVM compilers…

Material

- Books
  - Dragon Book
  - Advanced Compiler Design & Impl
  - Engineering a Compiler
- Blogs
  - V8, JSCore

